

importC

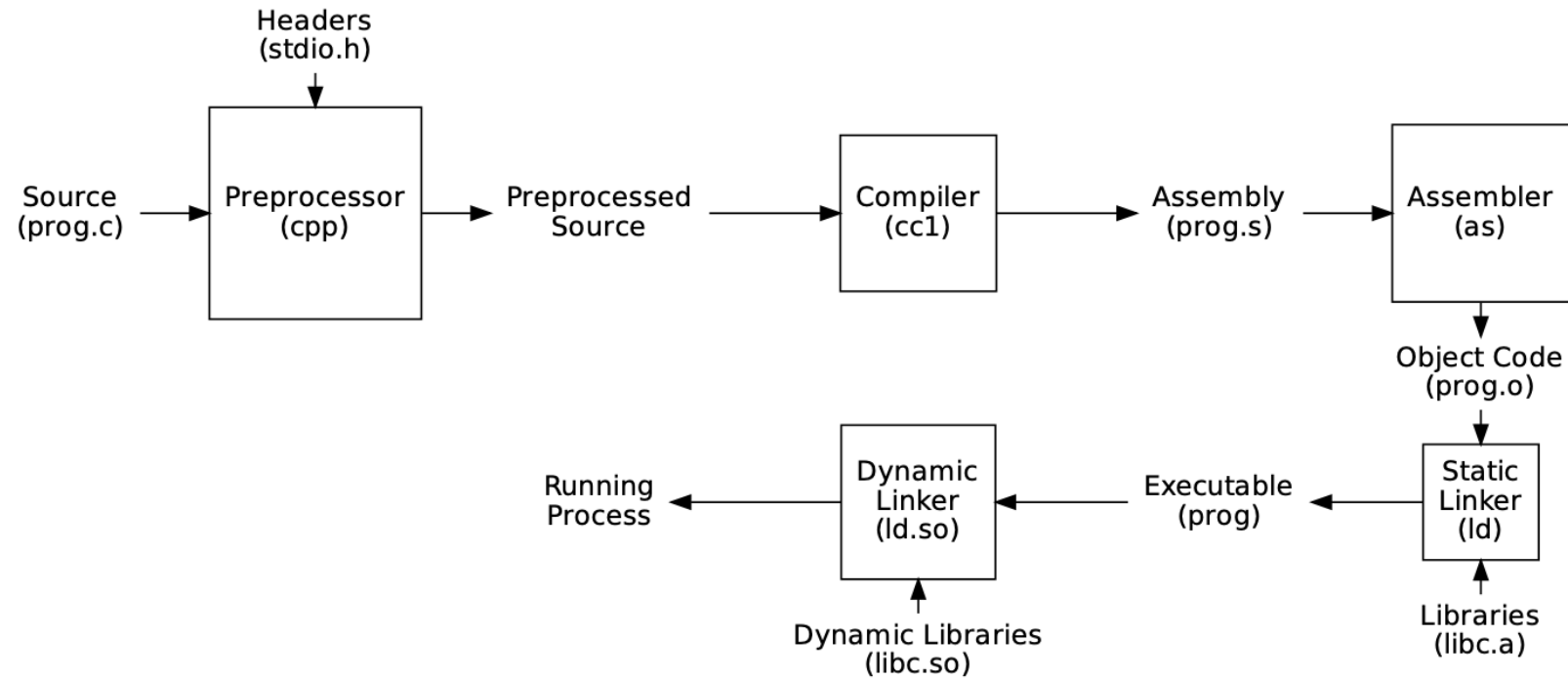
About me

- Compilers
- Operating systems
 - Networking
 - RAID (storage systems)
 - device drivers
- Code Security research.

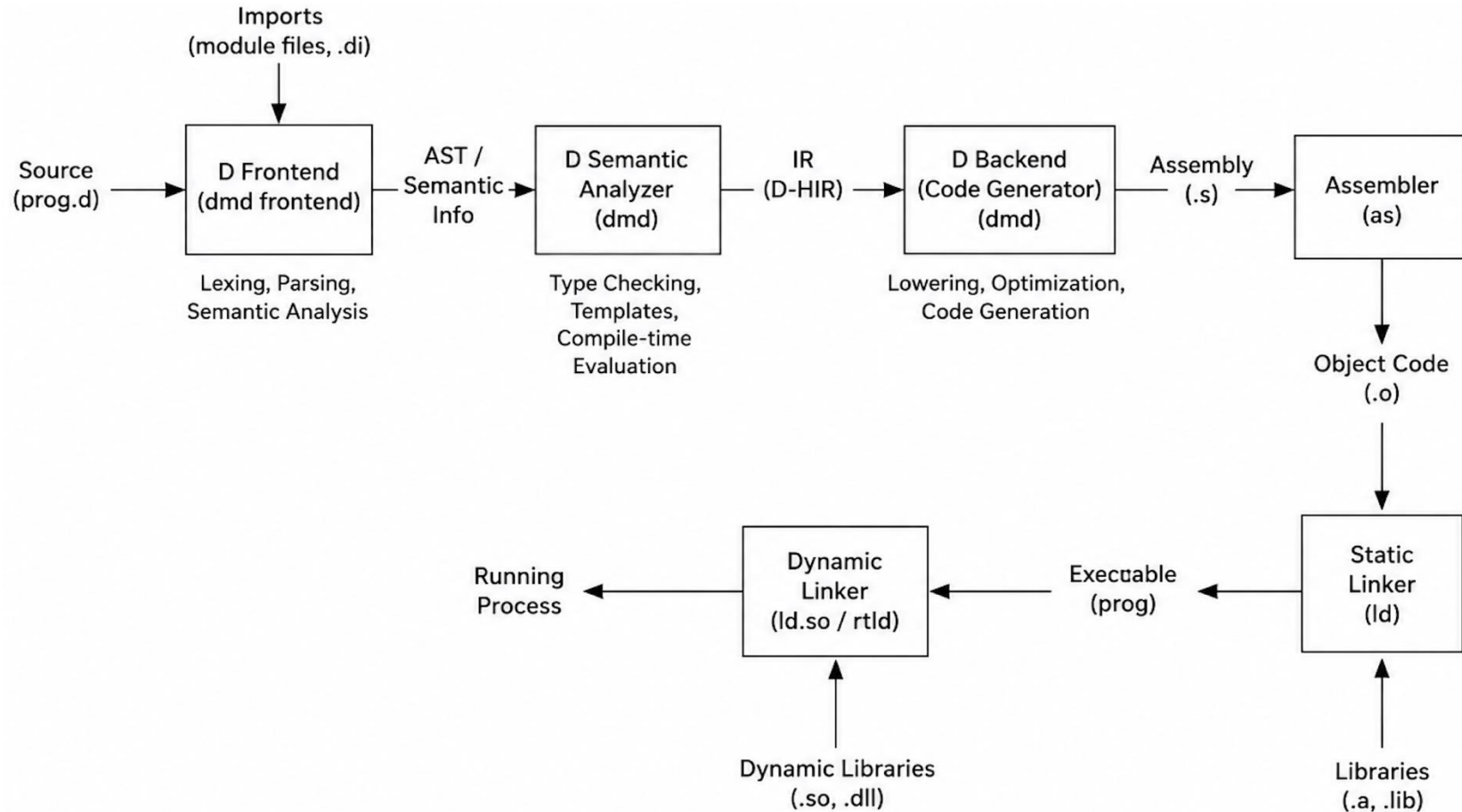
Purpose of this talk

- Another platform to tell the world about importC
- Talk about the most recent improvements

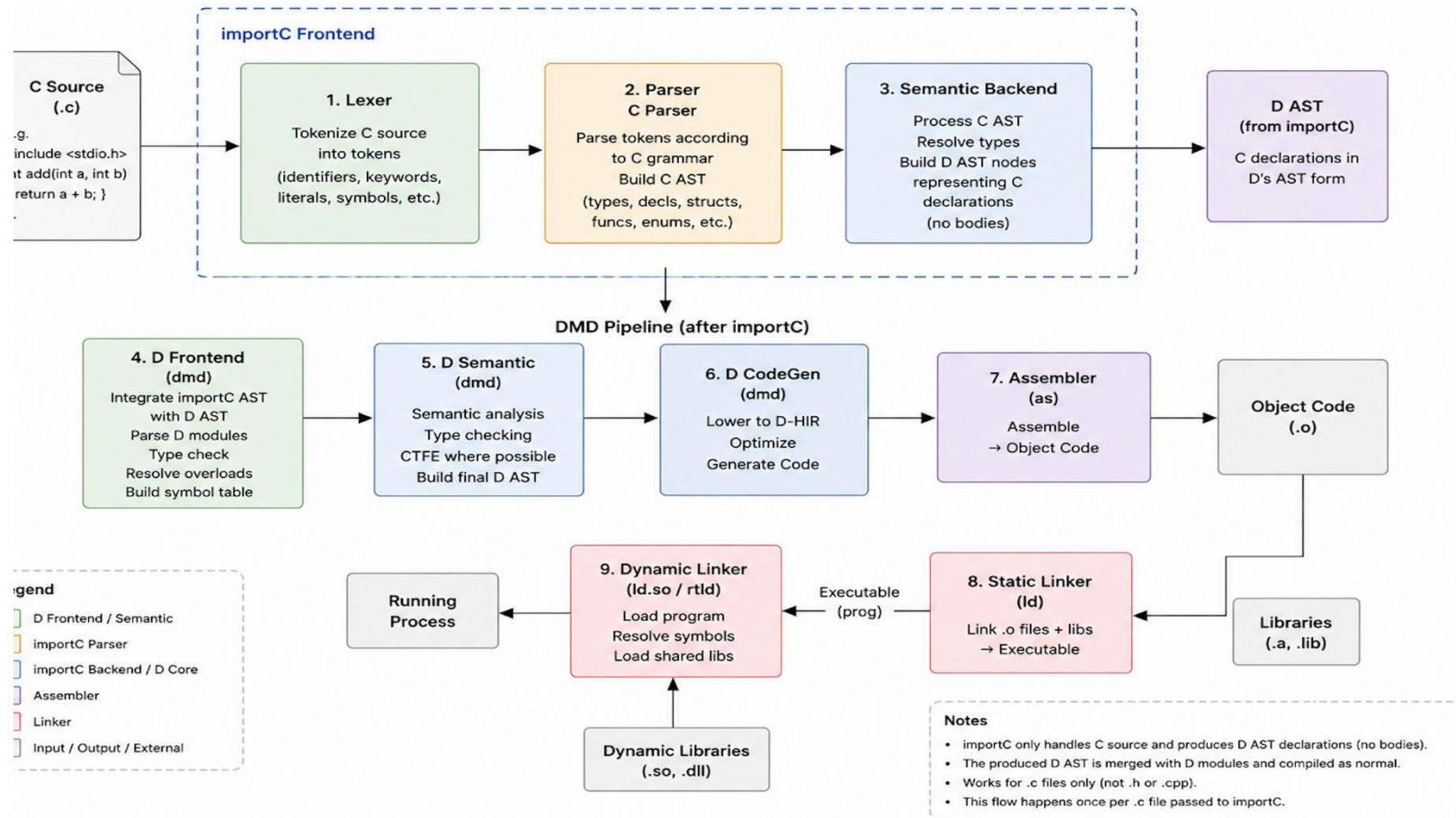
Basic C compiler mechanism



Basic D compiler mechanism



Compiler mechanism with importC



importC duality (use cases)

- Import a C module(source) from a D source and compile.
- Compile a C source directly

cFile.c

```
void cfunc()  
{  
    return;  
}
```

dfile.d

```
import cfile
```

```
void main() {  
    cfunc();  
}
```

```
dmd dfile.d cfile.c
```

```
./dfile //produce an executable “dfile”  
calls the C function cfunc().
```

Compiling the C source directly

```
// in cfile.c
void cfunc()
{
    return;
}
int main()
{
    cfunc();
    return 0;
}
```

```
dmd cfile.c
./cfile
```

The Honest Truth

- C is not going away anytime soon. (maybe never).
- 98 percent of our operating systems are still in C.
 - 1.5 percent because of rust in the Linux kernel.
 - 0.4 percent because of ongoing experiments for rust in the FreeBSD kernel.
 - 0.1 percent for those that I am not aware of yet.
- Legacy embedded device drivers are still in C.
- D having a compilable ability in these systems is key for adoption.

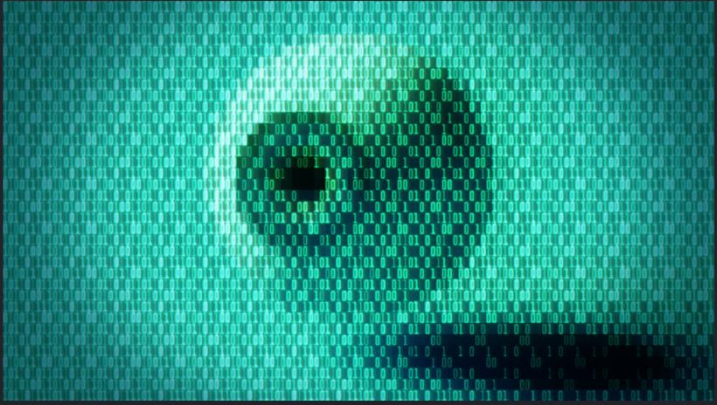
What I use importC for

 BEWARE OF BLANK LINES AND WHITE SPACES

Supply-chain attack using invisible code hits GitHub and other repositories

Unicode that's invisible to the human eye was largely abandoned—until attackers took notice.

DAN GOODIN · MAR 13, 2026 2:18 PM | 78



Credit: Aurich Lawson

Researchers say they've discovered a supply-chain attack flooding repositories with malicious packages that contain invisible code, a technique that's flummoxing traditional defenses designed to detect such threats.

The researchers, from firm Aikido Security, said Friday that they found 151 malicious packages that were uploaded to GitHub from March 3 to March 9. Such supply-chain attacks have been common for nearly a decade. They usually work by uploading malicious packages with code and names that closely resemble those of widely used code libraries, with the objective of tricking developers into mistakenly incorporating the former into their software. In some cases, these malicious packages are downloaded thousands of



Switch to Google Chrome

Browse securely with

What I use importC for

- I happen to manage servers located in different parts of the world who kind of know each other.
- They run a whole lot of different services. (mail, web, files etc.)
- Majority runs NetBSD with a few running Linux and FreeBSD.
- Sometimes, there be need for file relocations over the internet between machines that are miles away from each other.
- After a successful relocation, I run dmd on all simple C source files.
- For sanity checks. (There are bad guys on the internet).

After a simple copy and paste

```
long a =1000000, b = 4, c = 7, d, e, f = 8;  
int main(){  
    while (1){  
        c+=f;  
    }  
}
```

After a simple copy and paste

```
2 warnings generated.  
[joe@Emmanuels-MacBook-Pro ~ % vi tt.c  
[joe@Emmanuels-MacBook-Pro ~ % dmd -c ttt.c  
ttt.c(4): Error: character 0xa0 is not a valid token  
    c+=f;  
    ^  
ttt.c(4): Error: character 0xa0 is not a valid token  
    c+=f;  
    ^
```

importC improvements.

- Main Barrier
 - What C supports and D does not
 - What D supports and C does not

importC improvements.

- Designated Initializers
- Compound Literals
- Function and variable redeclarations
- Macros
- GCC Cyclic Redundancy Check (CRC) builtins
- DMD backend symbol duplications
- Static Linking
- Complex numbers

Symbol duplication (only on DMD)

cfile.c

// global declarations

int a;

int a;

int a;

dmd -c cfile.c

nm cfile.o

Symbol duplication (only on DMD)

```
nm cfile.0
```

```
00000000000000000000 s EH_frame0
```

```
00000000000000000000 B _a
```

```
00000000000000000004 B _a
```

```
00000000000000000008 B _a
```

Symbol duplication (only on DMD)

- Because It worked on LDC, I knew it was a backend problem.
- I have to go and change things in DMD's backend.
- After a week, trying to change things in DMD backend, I knew I had to go back to the frontend for my peace!

Symbol duplication (only on DMD)

cfile.c (Transformed into)

```
// global declarations
```

```
int a;
```

```
int a; ➔ static int a;
```

```
int a; ➔ static int a;
```

```
dmd -c cfile.c
```

```
nm cfile.o
```

Symbol duplication (only on DMD)

nm cfile.0 (fixed)

000000000000000000 s EH_frame0

000000000000000000 B _a

Designated Initializers

```
struct Foo
{
    int x;
    int y;
};
struct Bar
{
    struct Foo f;
};
struct Bar b = {
    .f.x = 3
}; // Error: only 1 designator currently allowed
```

Designated Initializers

```
struct Foo
{
    int x;
    int y;
};
struct Bar
{
    struct Foo f;
};
struct Bar b = {
    .f.x = 3
}; // now compiles
```

Taking address of compound Literals

```
int *p = &(int){8880}; // global
```

```
Int main() {  
    int *b = &(int){55};  
}
```

```
// Error: `[8880]` is not an lvalue and cannot be modified
```

Taking address of compound Literals

```
int *p = &(int){8880}; // global
```

```
Int main() {  
    int *b = &(int){55};  
}
```

creates a temporary variable and stores the literal initializer.
Then pointer points to it and used in the lifetime of the pointer.

Taking address of compound Literals

```
int *p = &(int){8880}; // global
```

```
Int main() {  
    int *b = &(int){55};  
}
```

```
// Compiles and run
```

```
assert(*p == 8880);
```

```
assert(*b == 55);
```

Variable redeclarations

// all in global scope

Allowed in C

```
extern int x;
```

```
extern int x;
```

Disallowed in C

```
extern int x;
```

```
extern int c;
```

Variable redeclarations

Strictly bound to D semantics (Breaks it)

```
extern int x;
```

```
extern int x;
```

Variable redeclarations

Has been freed from bondage of D semantics

```
extern int x;
```

```
extern int x;
```

// new declarations replace previous declarations

// Compiles

Function redeclaration

```
void test()  
{  
    extern int f();  
    int f();  
}
```

Breaks D semantics(declaration of f() is already defined)

Function redeclaration

```
void test()  
{  
    extern int f();  
    int f();  
}
```

// Freed from the bondage of D semantics

// compiles

C Macros

- UNIX “IFNAMSIZ”
- The maximum string length of a network interface.

C Macros

- `#define IFNAMSIZ`
- `#define IFNAMSIZ 5`
- `#define IFNAMSIZ 4 + 5`
- `#define IFNAMSIZ "seven"`
- `#define IFNAMSIZ getifnamesize()`
- `#define IFNAMSIZ (5+6)`
- `#ifdef IFNAMSIZ`
- `#ifndef IFNAMSIZ`
- `#if defined(IFNAMSIZ)`
- `#endif`
- `#define IFNAMSIZ seven`

C Macros

```
#define IFNAMSIZ
```

D parser expands to nothing.

C Macros

```
#define IFNAMSIZ      5
```

```
enum IFNAMSIZ      = 5;
```

C Macros

```
#define IFNAMSIZ      5 + 7
```

```
auto IFNAMSIZ()()      { return 5 + 7; }
```

C Macros

```
#define IFNAMSIZ      “seven”
```

```
enum IFNAMSIZ  =      “seven”;
```

C Macros

```
#define IFNAMSIZ      (5 + 6)
```

```
auto IFNAMSIZ()()      { return 5 + 6; }
```

C Macros

Special case (identifier based)

```
#define IFNAMSIZ      seven
```

alias IFNAMSIZ = seven (Naïve solution)

C Macros

alias IFNAMSIZ = seven (Naive solution)

- Major troubles in C

- C → #define IFNAMSIZ IFNAMSIZ (valid in C)

- D → alias IFNAMSIZ = IFNAMSIZ (self invalid in D)

C Macros

alias IFNAMSIZ = seven (Naive solution)

- Major troubles in C

- C ➔ #define IFNAMSIZ UNKNOWN_SYMBOL (valid in C)

- D ➔ alias IFNAMSIZ = UNRESOLVABLE_SYMBOL (invalid in D)

C Macros

alias IFNAMSIZ = seven (Naive solution)

- Self aliasing
- Aliasing an unknown symbol.

C Macros

alias IFNAMSIZ = seven (Naive solution)

- The temporary good news is that, this could be easily handled in Semantic analysis.

```
if (importC && sc.inCfile) {  
    // don't cry  
    // just end semantics analysis for that symbol.  
}
```

C Macros

- What about function calls

```
#define IFNAMSIZ      getifnamesize(args)
```

C Macros

```
#define IFNAMSIZ      getifnamesize(args)
```

Just create a variable and initialize it with `getifnamesize` CallExp.

C Macros

```
#define IFNAMSIZ      getifnamesize(args)
```

```
var variable = getifnamesize()
```

- needs a return type
- getifnamesize's function body must be resolvable

C Macros

- The temporary bad news is when compiling a library.

Compiling a.c b.c d.c e.c f.c

C Macros

```
#define IFNAMSIZ      getifnamesize(args)
```

- C includes always leads to a lot of defines and declarations in source files after preprocessing.
- A macro declaration may have found its way into a C source via includes which may not necessarily need to be there.
- When D tries to resolve, It does in the symbol scope of the current compilation unit which then does not find the definition for the aliased symbol

C Macros

Special case

```
#define IFNAMSIZ      seven
```

```
auto IFNAMSIZ()()      { return seven; }
```

C Macros

Special case

- So just know that DMD frontend is creating template functions for your identifier macros(function calls, direct identifiers, and braced identifiers).
- via the ``addNullaryTemplate()`` function in DMD.

C Macros

```
#undef IFNAMSIZ
```

remove symbolized IFNAMSIZ from C scope symbol table.

Forward declarations with static libraries

```
static void static_fun();
```

```
void lib_fun()  
{  
    static_fun();  
}
```

```
static void static_fun()  
{  
}
```

```
// compiling with -lib was problematic
```

Forward declarations with static libraries

- The problem was that the compiler was not updating the symbol with the definitions properly.
- so the compiler only saw the first declaration in backend.
- So now, we transfer full definition to the first declaration and then updated in symbol table.

Forward declarations with static libraries

```
static void static_fun();
```

```
void lib_fun()  
{  
    static_fun();  
}
```

```
static void static_fun()  
{  
}
```

```
// compiling with -lib works!
```

Complex numbers

- Complex numbers are heavily used in control systems, signal processing, graphics, and scientific computing.
- C has been the language of choice for writing libraries used in high-performance DSP, embedded systems, and real-time signal pipelines.
- we had a hard time compiling C complex numbers in ImportC

Complex numbers

- Complex numbers are heavily used in control systems, signal processing, graphics, and scientific computing.
- C has been the language of choice for writing libraries used in high-performance DSP, embedded systems, and real-time signal pipelines.
- we had a hard time compiling C complex numbers in ImportC

Complex numbers

- Complex numbers are heavily used in control systems, signal processing, graphics, and scientific computing.
- C has been the language of choice for writing libraries used in high-performance DSP, embedded systems, and real-time signal pipelines.
- we had a hard time compiling C complex numbers in ImportC

Complex numbers

- Complex types were entirely using the “new” complex library struct.
- D was heading towards a direction to deprecate the fundamental complex type (As Far As I know).
- But we need to revert if we care about importC
- Currently, the fundamental complex type has regained some momentum for import C in the compiler.

Complex numbers

```
#include <complex.h>
#include <stdio.h>
#include <assert.h>
struct com
{
    int r;
    int c;
};
void foo() {
    struct com obj = { 1, 3 };
    _Complex double x = { obj.r, obj.c };
    // real + im*i assert(creal(x) == 1.000000); assert(cimag(x) == 3.000000); return;
}
```

IMPORTC

We hope these recent fixes will improve your importC usage!

THANK YOU!